

**Mathematical Programming and
Operations Research**
Modeling, Algorithms, and Complexity
Examples in Excel and Python
(Work in progress)

Edited by: Robert Hildebrand

Contributors: Robert Hildebrand, Laurent Poirrier, Douglas Bish, Diego Moran

Version Compilation date: March 24, 2025

Preface

This entire book is a working manuscript. The first draft of the book is yet to be completed.

This book is being written and compiled using a number of open source materials. We will strive to properly cite all resources used and give references on where to find these resources. Although the material used in this book comes from a variety of licences, everything used here will be CC-BY-SA 4.0 compatible, and hence, the entire book will fall under a CC-BY-SA 4.0 license.

MAJOR ACKNOWLEDGEMENTS

I would like to acknowledge that substantial parts of this book were borrowed under a CC-BY-SA license. These substantial pieces include:

- "A First Course in Linear Algebra" by Lyryx Learning (based on original text by Ken Kuttler). A majority of their formatting was used along with selected sections that make up the appendix sections on linear algebra. We are extremely grateful to Lyryx for sharing their files with us. They do an amazing job compiling their books and the templates and formatting that we have borrowed here clearly took a lot of work to set up. Thank you for sharing all of this material to make structuring and forming this book much easier! See subsequent page for list of contributors.
- "Foundations of Applied Mathematics" with many contributors. See <https://github.com/Foundations-of-Applied-Mathematics>. Several sections from these notes were used along with some formatting. Some of this content has been edited or rearranged to suit the needs of this book. This content comes with some great references to code and nice formatting to present code within the book. See subsequent page with list of contributors.
- "Linear Inequalities and Linear Programming" by Kevin Cheung. See <https://github.com/dataopt/lineqlpbook>. These notes are posted on GitHub in a ".Rmd" format for nice reading online. This content was converted to $\text{\LaTeX}$ using Pandoc. These notes make up a substantial section of the Linear Programming part of this book.
- Linear Programming notes by Douglas Bish. These notes also make up a substantial section of the Linear Programming part of this book.

I would also like to acknowledge Laurent Porrier and Diego Moran for contributing various notes on linear and integer programming.

I would also like to thank Jamie Fravel for helping to edit this book and for contributing chapters, examples, and code.

Contents

Contents	5
1 Resources and Notation	5
2 Mathematical Programming	9
2.1 Warm-Up Scenario	9
2.2 Why Study Operations Research?	11
2.3 What is Mathematical Programming?	11
2.4 Applications	12
2.5 Types of Optimization problems	13
2.6 Linear Programming (LP)	13
2.7 Mixed-Integer Linear Programming (MILP)	14
2.8 Non-Linear Programming (NLP)	16
2.8.1 Convex Programming	17
2.8.2 Non-Convex Non-linear Programming	17
2.8.3 Machine Learning	18
2.9 Mixed Integer Non-Linear Programming (MINLP)	19
2.9.1 Convex MINLP	20
2.9.2 Non-Convex MINLP	20
2.10 Optimization Under Uncertainty	23
2.10.1 Stochastic Optimization	23
2.10.2 Robust Optimization	23
2.10.3 Applications	23
I Linear Programming	25
3 Modeling: Linear Programming	27
3.1 Introductory LP Example	28
3.2 Modeling and Assumptions in Linear Programming	29
3.2.1 Assumptions	31
3.3 Examples	32
3.3.1 Knapsack Problem	40
3.3.2 Capital Investment	41
3.3.3 Assignment Problem	42
3.4 Exercises	43
4 Software - Excel	45

4.1	Using Excel Solver	45
4.1.1	Useful Excel Functions and Techniques	45
4.1.2	How to Use Excel Solver	47
5	Graphically Solving Linear Programs	51
5.1	Nonempty and Bounded Problem	51
5.2	Graphical Approach	53
5.3	Infinitely Many Optimal Solutions	57
5.4	Problems with No Solution	59
5.5	Problems with Unbounded Feasible Regions	60
5.6	Extreme Directions	67
5.7	Exercises	69
	Solutions	70
6	Modeling with Compact Notation	73
6.1	Production Planning Models	74
6.2	Assignment Problem	78
6.3	Assignment Problem	81
6.4	Modeling Tricks	87
6.4.1	Maximizing a minimum	87
6.5	Network Flow Models and Applications	88
6.5.1	Graphs	89
6.5.1.1	Undirected Graphs	89
6.5.1.2	Directed Graphs	89
6.5.2	Example: Minimum-Cost Flow in a Transportation Network	90
6.5.3	Minimum Cost Network Flow Problem	92
6.5.4	Maximum flow	94
7	Formal Mathematical Statements	101
7.0.1	Vectors and Their Properties	101
7.0.2	Linear Combinations and Related Concepts	102
7.1	Convexity	103
7.1.1	Convex Sets	103
7.1.2	Spanning Sets and Basis	105
7.1.3	Convex and Polyhedral Sets	106
7.1.4	Unbounded Sets and Rays	109
8	Simplex Method	111
8.1	Standard Form	111
8.1.1	Standard Form for Constraints	112
8.1.2	Converting a Minimization Problem to a Maximization Problem	115
8.1.3	Exercises: Converting to Standard Form	116
8.2	Solutions to $A\mathbf{x} = \mathbf{b}$	118
8.2.1	Partitioning the Variables	119

8.2.2	Another Example	121
8.2.3	Selecting and Evaluating Different Bases	122
8.3	Optimality Conditions	123
8.4	Pivoting and the Simplex Algorithm	126
8.4.1	Transforming the LP to Standard Form	127
8.4.2	Summarizing the steps taken	133
8.5	The Simplex Algorithm!	134
8.6	No Feasible Initial Basis and the Big-M Method	138
8.6.1	Let's see another example	145
8.7	Degeneracy, Cycling, and Bland's Rule	146
8.7.1	Degeneracy and Unchanged BFS after Pivoting	147
8.7.2	Cycling in the Simplex Method	148
8.7.3	Bland's Rule	149
9	Sensitivity Analysis	151
9.0.1	Mathematical details of sensitivity analysis	151
9.1	Sensitivity Analysis in the Revised Simplex Tableau	152
9.1.1	Effect of Changing an Objective Coefficient c_i	153
9.1.2	Effect of Changing the Right-Hand Side $\mathbf{b}$	153
9.1.3	Effect of Changing the Coefficients a_{ij} of the Constraint Matrix A	154
9.1.4	Summary Table: Sensitivity Effects in the Revised Simplex Tableau	155
9.1.5	Computing the Range of Parameters That Maintain the Optimal Basis	155
9.1.5.1	Original Problem and Optimal Tableau	155
9.1.5.2	1. Changing the Right-Hand Side b_1 of the First Constraint	156
9.1.5.3	Sensitivity Analysis on the Objective Coefficient c_x	157
9.1.6	Excel Solver Sensitivity Analysis	158
10	Duality	161
10.1	Duality Theory	161
10.2	Primal-Dual Pairs and Strong Duality	164
II	Discrete Algorithms	171
11	Graph Algorithms	173
11.1	Graph Theory and Network Flows	173
11.2	Graphs	173
11.2.1	Drawing Graphs	173
11.3	Definitions	176
11.4	Shortest Path	179
11.5	Spanning Trees	188
11.6	Exercise Answers	192
11.7	Prim's Algorithm	194
11.8	Additional Exercises	194

11.8.1	Notes	203
A	Linear Equations	205
A.1	Graphing a Linear Equation	205
A.2	Slope of a Line	209
A.3	Determining the Equation of a Line	211
A.4	Applications of Linear Equations	212
A.5	More Applications: Systems of Linear Equations	213
B	Systems of Equations	217
B.1	Systems of Equations, Geometry	217
B.2	Systems Of Equations, Algebraic Procedures	221
B.2.1	Elementary Operations	223
B.2.2	Gaussian Elimination	227
B.2.3	Uniqueness of the Reduced Row-Echelon Form	239
B.2.4	Rank and Homogeneous Systems	241
C	Matrices	247
C.1	Matrix Arithmetic	247
C.1.1	Addition of Matrices	249
C.1.2	Scalar Multiplication of Matrices	251
C.1.3	Multiplication of Matrices	252
C.1.4	The ij^{th} Entry of a Product	259
C.1.5	Properties of Matrix Multiplication	262
C.1.6	The Transpose	263
C.1.7	The Identity and Inverses	265
C.1.8	Finding the Inverse of a Matrix	267
D	$\mathbb{R}^n$	273
D.1	Vectors in $\mathbb{R}^n$	273
D.2	Algebra in $\mathbb{R}^n$	276
D.2.1	Addition of Vectors in $\mathbb{R}^n$	277
D.2.2	Scalar Multiplication of Vectors in $\mathbb{R}^n$	278
D.3	Geometric Meaning of Vector Addition	279
D.4	Length of a Vector	282
D.5	Geometric Meaning of Scalar Multiplication	286
D.6	The Dot Product	288
D.6.1	The Dot Product	288
D.6.2	The Geometric Significance of the Dot Product	291
D.7	Contributors	295

Introduction

Letter to instructors

This is an introductory book for students to learn optimization theory, tools, and applications. The two main goals are (1) students are able to apply the of optimization in their future work or research (2) students understand the concepts underlying optimization solver and use this knowledge to use solvers effectively.

This book was based on a sequence of course at Virginia Tech in the Industrial and Systems Engineering Department. The courses are *Deterministic Operations Research I* and *Deterministic Operations Reserach II*. The first course focuses on linear programming, while the second course covers integer programming an nonlinear programming. As such, the content in this book is meant to cover 2 or more full courses in optimization.

The book is designed to be read in a linear fashion. That said, many of the chapters are mostly independent and could be rearranged, removed, or shortened as desited. This is an open source textbook, so use it as you like. You are encouraged to share adaptations and improvements so that this work can evolve over time.

Letter to students

This book is designed to be a resource for students interested in learning what optimizaiton is, how it works, and how you can apply it in your future career. The main focus is being able to apply the techniques of optimization to problems using computer technology while understanding (at least at a high level) what the computer is doing and what you can claim about the output from the computer.

Quite importantly, keep in mind that when someone claims to have *optimized* a problem, we want to know what kind of gaurantees they have about how good their solution is. Far too often, the solution that is provided is suboptimal by 10%, 20%, or even more. This can mean spending excess amounts of money, time, or energy that could have been saved. And when problems are at a large scale, this can easily result in millions of dollars in savings.

For this reason, we will learn the perspective of *mathematical programming* (a.k.a. mathematical optimization). The key to this study is that we provide gaurantees on how good a solution is to a given problem. We will also study how difficult a problem is to solve. This will help us know (a) how long it might take to solve it and (b) how good a of a solution we might expect to be able to find in reasonable amount of time.

2 ■ CONTENTS

We will later study *heuristic* methods - these methods typically do not come with gaurantees, but tend to help find quality solutions.

Note: Although there is some computer programming required in this work, this is not a course on programming. Thanks to the fantastic modelling packages available these days, we are able to solve complicated problems with little programming effort. The key skill we will need to learn is *mathematical modeling*: converting words and ideas into numbers and variables in order to communicate problems to a computer so that it can solve a problem for you.

As a main element of this book, we would like to make the process of using code and software as easy as possible. Attached to most examples in the book, there will be links to code that implements and solves the problem using several different tools from Excel and Python. These examples can should make it easy to solve a similar problem with different data, or more generally, can serve as a basis for solving related problems with similar structure.

How to use this book

Skim ahead. We recommend that before you come across a topic in lecture, that you skim the relevant sections ahead of time to get a broad overview of what is to come. This may take only a fraction of the time that it may take for you to read it.

Read the expected outcomes. At the beginning of each section, there will be a list of expected outcomes from that section. Review these outcomes before reading the section to help guide you through what is most relevant for you to take away from the material. This will also provide a brief look into what is to follow in that section.

Read the text. Read carefully the text to understand the problems and techniques. We will try to provide a plethora of examples, therefore, depending on your understanding of a topic, you many need to go carefully over all of the examples.

Explore the resources. Lastly, we recognize that there are many alternative methods of learning given the massive amounts of information and resources online. Thus, at the end of each section, there will be a number of superb resources that available on the internet in different formats. There are other free textbooks, informational websites, and also number of fantastic videos posted to youtube. We encourage to explore the resources to get another perspective on the material or to hear/read it taught from a differnt point of view or in presentation style.

Outline of this book

This book is divided in to 4 Parts:

Part I Linear Programming,

Part II Integer Programming,

Part III Discrete Algorithms,

Part IV Nonlinear Programming.

There are also a number of chapters of background material in the Appendix.

The content of this book is designed to encompass 2-3 full semester courses in an industrial engineering department.

Work in progress

This book is still a work in progress, so please feel free to send feedback, questions, comments, and edits to Robert Hildebrand at open.optimization@gmail.com.

DISCRETE MATHEMATICS, GRAPHS, ALGORITHMS, AND COMBINATORICS

- [Levin - Discrete Mathematics - An Open Introduction, 3rd edition](#)
- [Github - Discrete Mathematics: an Open Introduction](#) CC BY SA
- [Keller, Trotter - Applied Combinatorics](#) (CC-BY-SA 4.0)
- [Keller - Github - Applied Combinatorics](#)

MIXED INTEGER NONLINEAR PROGRAMMING

- [Mixed-Integer Nonlinear Optimization](#) Pietro Belotti, Christian Kirches, Sven Leyffer, Jeff Linderoth, Jim Luedtke, and Ashutosh Mahajan

PROGRAMMING WITH PYTHON

- [A Byte of Python](#)
- [Github - Byte of Python](#) (CC-BY-SA)

Also, go to <https://github.com/open-optimization/open-optimization-or-examples> to look at more examples.

Notation

- $\mathbf{1}$ - a vector of all ones (the size of the vector depends on context)
- $\forall$ - for all
- $\exists$ - there exists
- $\in$ - in
- $\therefore$ - therefore
- $\Rightarrow$ - implies
- s.t. - such that (or sometimes "subject to".... from context?)
- $\{0, 1\}$ - the set of numbers 0 and 1
- $\mathbb{Z}$ - the set of integers (e.g. $1, 2, 3, -1, -2, -3, \dots$)
- $\mathbb{Q}$ - the set of rational numbers (numbers that can be written as p/q for $p, q \in \mathbb{Z}$ (e.g. $1, 1/6, 27/2$))
- $\mathbb{R}$ - the set of all real numbers (e.g. $1, 1.5, \pi, e, -11/5$)
- $\setminus$ - setminus, (e.g. $\{0, 1, 2, 3\} \setminus \{0, 3\} = \{1, 2\}$)
- $\cup$ - union (e.g. $\{1, 2\} \cup \{3, 5\} = \{1, 2, 3, 5\}$)

- $\cap$ - intersection (e.g. $\{1, 2, 3, 4\} \cap \{3, 4, 5, 6\} = \{3, 4\}$)
- $\{0, 1\}^4$ - the set of 4 dimensional vectors taking values 0 or 1, (e.g. $[0, 0, 1, 0]$ or $[1, 1, 1, 1]$)
- $\mathbb{Z}^4$ - the set of 4 dimensional vectors taking integer values (e.g., $[1, -5, 17, 3]$ or $[6, 2, -3, -11]$)
- $\mathbb{Q}^4$ - the set of 4 dimensional vectors taking rational values (e.g. $[1.5, 3.4, -2.4, 2]$)
- $\mathbb{R}^4$ - the set of 4 dimensional vectors taking real values (e.g. $[3, \pi, -e, \sqrt{2}]$)
- $\sum_{i=1}^4 i = 1 + 2 + 3 + 4$
- $\sum_{i=1}^4 i^2 = 1^2 + 2^2 + 3^2 + 4^2$
- $\sum_{i=1}^4 x_i = x_1 + x_2 + x_3 + x_4$
- $\square$ - this is a typical Q.E.D. symbol that you put at the end of a proof meaning "I proved it."
- For $x, y \in \mathbb{R}^3$, the following are equivalent (note, in other contexts, these notations can mean different things)

– $x^\top y$ *matrix multiplication*

– $x \cdot y$ *dot product*

– $\langle x, y \rangle$ *inner product*

and evaluate to $\sum_{i=1}^3 x_i y_i = x_1 y_1 + x_2 y_2 + x_3 y_3$.

A sample sentence:

$$\forall x \in \mathbb{Q}^n \exists y \in \mathbb{Z}^n \setminus \{0\}^n \text{ s.t. } x^\top y \in \{0, 1\}$$

"For all non-zero rational vectors x in n -dimensions, there exists a non-zero n -dimensional integer vector y such that the dot product of x with y evaluates to either 0 or 1."

2. Mathematical Programming

Outcomes

- *Consider an introductory problem*
- *Identify reasons for studying operations research*
- *Define "Mathematical Programming"*
- *Learn about different applications of the tools in this book*
- *Explore the different types of optimization models and what types we will see in this book*

2.1 Warm-Up Scenario

Imagine you're starting a new clothing company and decide to sell shirts. You've opened a store in Roanoke and, based on market research, you estimate the demand for the upcoming weekend:

- **Friday:** 5 shirts
- **Saturday:** 10 shirts
- **Sunday:** 7 shirts

Your supplier can deliver shirts in one day, meaning an order placed on Thursday will arrive on Friday, and so on. As a business owner, you need to decide how many shirts to order and when to place each order to ensure you meet the daily demand without overstocking.

Defining the Problem

To tackle this challenge, you'll need to think strategically about two main factors:

- **Ordering Quantities:** How many shirts to order on each day.
- **Inventory Levels:** How many shirts to keep in stock at the end of each day.

You can define these decisions using variables:

- Let the number of shirts you order on a specific day be a decision variable. For example, you could track the number of shirts ordered on Thursday, Friday, and Saturday.
- Define variables to represent the number of shirts you have in stock at the end of each day.

Your goal is to meet the projected demand while managing costs. Costs could include:

- The expense of ordering shirts.
- The cost of holding inventory.
- Penalties for running out of stock.

Refining the Scenario

As your business grows, additional complexities may arise. For instance:

- The cost of ordering shirts might vary by day due to supplier pricing.
- Holding inventory could incur storage costs.
- A budget might limit how much you can spend on orders.
- You might expand your product line to include pants and socks, each with its own demand and costs.

By clearly defining variables and constraints for these decisions, you can systematically approach the problem and develop an optimal strategy. This step-by-step process of defining the problem, incorporating real-world details, and refining the solution is a fundamental aspect of optimization.

Model Development Cycle

This multi-stage example illustrates the iterative cycle of modeling:

1. **Initial Problem:** Determine ordering quantities to meet demand.
2. **Modeling:** Formulate an optimization model based on available information.
3. **Solution:** Solve the model to find the optimal ordering plan.
4. **New Information:** Incorporate additional real-world complexities (varying costs, inventory costs, budget constraints, additional products, fixed ordering costs).
5. **Refinement:** Update the model to reflect new information and solve again.

Further Extensions

Looking ahead, there are several ways to expand this scenario:

- **Expanding to New Stores:** Opening additional stores introduces location-based demand, transportation costs, and coordination of inventory across multiple sites.
- **Hiring New Staff:** Incorporate labor costs, scheduling constraints, and productivity rates into the model.

- **Producing Clothing In-House:** Transitioning to manufacturing involves modeling production processes, including purchasing raw materials, machine capacities, and processing times.
- **Uncertain Demand:** In reality, demand may be uncertain. Advanced models can incorporate stochastic elements or use probabilistic methods to handle randomness.

In this course, we will focus on deterministic models where all data is known and fixed. Building a strong foundation with these models will prepare us to tackle more complex problems involving uncertainty and randomness in future studies.

2.2 Why Study Operations Research?

Operations Research (OR) is a discipline that applies advanced analytical methods to help make better decisions. In a world where resources are limited and organizations face complex challenges, OR provides the tools and methodologies to model, analyze, and solve problems involving the optimal allocation of scarce resources. Studying OR equips individuals with a systematic and quantitative approach to decision-making, which is invaluable in industries such as logistics, finance, healthcare, and manufacturing.

By understanding OR, you gain the ability to construct mathematical models of real-world systems, allowing for the simulation and optimization of different scenarios. This leads to improved efficiency, reduced costs, and enhanced performance. Moreover, the skills developed through studying OR—such as critical thinking, problem-solving, and data analysis—are highly transferable and sought after in today's data-driven economy.

Operations Research also bridges the gap between theory and practice. It transforms abstract mathematical concepts into actionable strategies that can have a tangible impact on organizational success. Whether it's optimizing supply chains, scheduling flights, or managing portfolios, the applications of OR are vast and impactful.

2.3 What is Mathematical Programming?

Mathematical Programming is a core area within Operations Research that focuses on the formulation and solution of optimization problems. It involves creating mathematical models that represent the decision-making process of a system, where the goal is to find the best possible outcome under given constraints. The term "programming" in this context refers to planning or scheduling, not computer programming.

At the heart of Mathematical Programming is the objective function—a mathematical expression that defines the criterion to be optimized, such as minimizing costs or maximizing profits. The decision variables represent the choices available, and the constraints reflect the limitations or requirements of the problem, such as resource capacities or regulatory conditions.

There are various types of mathematical programming, each suited to different kinds of problems:

- **Linear Programming (LP):** Deals with problems where both the objective function and constraints are linear.
- **Integer Programming (IP):** Involves decision variables that must take on integer values.
- **Nonlinear Programming (NLP):** Handles problems with nonlinear objective functions or constraints.
- **Dynamic Programming (DP):** Breaks down problems into simpler subproblems and is particularly useful for multistage decision processes.

Mathematical Programming provides a powerful framework for optimizing complex systems and making informed decisions. It combines elements of mathematics, economics, and computer science to develop solutions that are both efficient and effective.

2.4 Applications

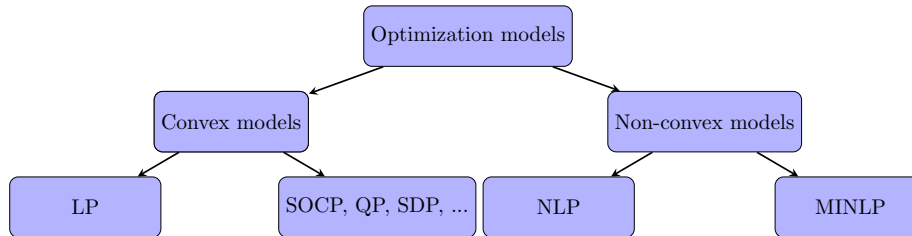
Operations Research and Mathematical Programming have a wide range of applications across various industries and sectors:

- **Transportation and Logistics:** Optimizing routing and scheduling for delivery vehicles, reducing transportation costs, and improving supply chain efficiency.
- **Manufacturing:** Streamlining production processes, managing inventory levels, and scheduling machinery to maximize productivity and minimize waste.
- **Finance:** Portfolio optimization, risk assessment, and capital budgeting to enhance financial performance and manage uncertainties.
- **Healthcare:** Allocating resources such as staff and equipment, scheduling surgeries, and managing patient flow to improve service quality and reduce wait times.
- **Energy Sector:** Optimizing power generation and distribution, planning for renewable energy integration, and managing energy consumption.
- **Telecommunications:** Network design and optimization, bandwidth allocation, and improving the reliability and efficiency of communication systems.
- **Agriculture:** Planning crop rotations, optimizing the use of fertilizers and water, and managing supply chains for agricultural products.
- **Public Sector:** Urban planning, emergency response logistics, and policy analysis to enhance public services and resource utilization.

These applications demonstrate the versatility and impact of OR and Mathematical Programming. By leveraging these tools, organizations can make data-driven decisions that lead to significant improvements in performance, cost savings, and competitive advantage. As global challenges become more complex, the importance of skilled professionals in Operations Research continues to grow.

2.5 Types of Optimization problems

We will state main general problem classes to be associated with in these notes. These are Linear Programming (LP), Mixed-Integer Linear Programming (MILP), Non-Linear Programming (NLP), and Mixed-Integer Non-Linear Programming (MINLP).



Along with each problem class, we will associate a complexity class for the general version of the problem. Complexity classes are discussed in a later portion of the book. Although we will often state that input data for a problem comes from $\mathbb{R}$ (as a real number), when we discuss complexity of such a problem, we actually mean that the data is rational, i.e., from $\mathbb{Q}$ (rational numbers that are encoded in the computer typically), and is given in binary encoding.

CAUTION!

In the following subsections, we will use advanced notation that may be unfamiliar at this point. These mathematical notations will be developed throughout the book.

Furthermore, don't be too concerned about the specific definitions of the complexity classes at this point. For now, you can read that *Polynomial time (P)* means it should be relatively easy to solve, while the other classes might be much more difficult to solve.

2.6 Linear Programming (LP)

Some linear programming background, theory, and examples will be provided in ??.

Linear Programming (LP):

Polynomial time (P)

Given a matrix $A \in \mathbb{R}^{m \times n}$, vector $b \in \mathbb{R}^m$ and vector $c \in \mathbb{R}^n$, the *linear programming* problem is

$$\begin{aligned}
 \max \quad & c^\top x \\
 \text{s.t.} \quad & Ax \leq b \\
 & x \geq 0
 \end{aligned} \tag{2.1}$$

Linear programming can come in several forms, whether we are maximizing or minimizing, or if the constraints are $\leq$, $=$ or $\geq$. One form commonly used is *Standard Form* given as

Linear Programming (LP) Standard Form:

Polynomial time (P)

Given a matrix $A \in \mathbb{R}^{m \times n}$, vector $b \in \mathbb{R}^m$ and vector $c \in \mathbb{R}^n$, the *linear programming* problem in *standard form* is

$$\begin{aligned} \max \quad & c^\top x \\ \text{s.t.} \quad & Ax = b \\ & x \geq 0 \end{aligned} \tag{2.2}$$

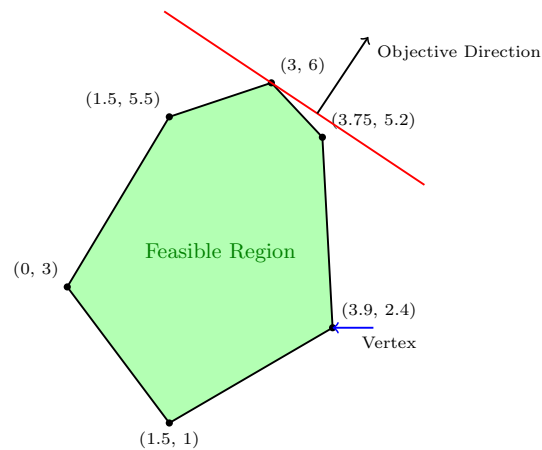


Figure 2.1: Linear programming constraints and objective.

2.7 Mixed-Integer Linear Programming (MILP)

Mixed-integer linear programming will be the focus of Sections ??, ??, ??, and ??. Recall that the notation $\mathbb{Z}$ means the set of integers and the set $\mathbb{R}$ means the set of real numbers. The first problem of interest here is a *binary integer program* (BIP) where all n variables are binary (either 0 or 1).

Part I

Linear Programming

3. Modeling: Linear Programming

Outcomes

1. Define what a linear program is
2. Understand how to model a linear program
3. View many examples and get a sense of what types of problems can be modeled as linear programs.

Linear Programming, also known as Linear Optimization, is the starting point for most forms of optimization. It is the problem of optimization a linear function over linear constraints.

In this chapter, we will define what this means, how to setup a linear program, and discuss many examples. Examples will be connected with code in Excel and Python (using with PuLP or Gurobipy modeling tools) so that you can easily start solving optimization problems. Tutorials on these tools will come in later chapters.

As a warm up, consider the following word problem.

Question: Find the age of Justin and Brad, given that the sum of their ages is equal to 76 and after 7 years, the age of Brad will be twice the age of Justin.

Solution: Let x and y denote the age of Justin and Brad, respectively. Then

$$x + y = 76 \tag{1}$$

$$2(x + 7) = y + 7. \tag{2}$$

On solving the foregoing equations we get

$$x = 23 \quad \text{and} \quad y = 53.$$

We will need to think about assigning variables to decisions and instead of equations guiding these variables, we may have inequalities that bound these variables in different ways. Thus, there can be many possible solutions.

3.1 Introductory LP Example

We begin with a simple example.

Example 3.1: Toy Maker

Excel PuLP Gurobipy

Consider the problem of a toy company that produces toy planes and toy boats. The toy company can sell its planes for \$10 and its boats for \$8 dollars. It costs \$3 in raw materials to make a plane and \$2 in raw materials to make a boat. A plane requires 3 hours to make and 1 hour to finish while a boat requires 1 hour to make and 2 hours to finish. The toy company knows it will not sell anymore than 35 planes per week. Further, given the number of workers, the company cannot spend anymore than 160 hours per week finishing toys and 120 hours per week making toys. The company wishes to maximize the profit it makes by choosing how much of each toy to produce.

We can represent the profit maximization problem of the company as a linear programming problem. Let x_1 be the number of planes the company will produce and let x_2 be the number of boats the company will produce. The profit for each plane is $\$10 - \$3 = \$7$ per plane and the profit for each boat is $\$8 - \$2 = \$6$ per boat. Thus the total profit the company will make is:

$$z(x_1, x_2) = 7x_1 + 6x_2 \quad (3.1)$$

The company can spend no more than 120 hours per week making toys and since a plane takes 3 hours to make and a boat takes 1 hour to make we have:

$$3x_1 + x_2 \leq 120 \quad (3.2)$$

Likewise, the company can spend no more than 160 hours per week finishing toys and since it takes 1 hour to finish a plane and 2 hour to finish a boat we have:

$$x_1 + 2x_2 \leq 160 \quad (3.3)$$

Finally, we know that $x_1 \leq 35$, since the company will make no more than 35 planes per week. Thus the complete linear programming problem is given as:

$$\begin{aligned} \max \quad & z(x_1, x_2) = 7x_1 + 6x_2 \\ \text{s.t.} \quad & 3x_1 + x_2 \leq 120 \\ & x_1 + 2x_2 \leq 160 \\ & x_1 \leq 35 \\ & x_1 \geq 0 \\ & x_2 \geq 0 \end{aligned} \quad (3.4)$$

Exercise 3.1: Chemical Manufacturing

A chemical manufacturer produces three chemicals: A, B and C. These chemical are produced by two processes: 1 and 2.

- Running process 1 for 1 hour costs \$4 and yields 3 units of chemical A, 1 unit of chemical B and 1 unit of chemical C.
- Running process 2 for 1 hour costs \$1 and produces 1 units of chemical A, and 1 unit of chemical B (but none of Chemical C).

To meet customer demand,

- at least 10 units of chemical A,
- at least 5 units of chemical B and
- at least 3 units of chemical C

must be produced daily.

Assume that the chemical manufacturer wants to minimize the cost of production.

Develop a linear programming problem describing the constraints and objectives of the chemical manufacturer.

[Hint: Let x_1 be the amount of time Process 1 is executed and let x_2 be amount of time Process 2 is executed. Use the coefficients above to express the cost of running Process 1 for x_1 time and Process 2 for x_2 time. Do the same to compute the amount of chemicals A, B, and C that are produced.]

3.2 Modeling and Assumptions in Linear Programming

Outcomes

1. Address crucial assumptions when chosing to model a problem with linear programming.

A Generic Linear Program (LP)

Linear programming (LP) is a mathematical optimization framework widely used in decision-making and resource allocation. In its most basic form, an LP seeks to optimize (maximize or minimize) a linear objective function subject to a set of linear constraints. These constraints typically represent limitations or requirements in real-world scenarios, such as budgets, capacities, or resource availability.

Decision Variables:

The decision variables represent the quantities we want to determine. For example:

- x_i : Continuous variables ($x_i \in \mathbb{R}$, i.e., a real number), $\forall i = 1, \dots, n$.

These variables could represent amounts of products to produce, resources to allocate, or investments to make. We can use any letters to represent these variables. Typically we use x_i , but frequently will use y_i or other letters depending on the context and what makes reading the problem most convenient.

Parameters (Known Input Parameters):

Parameters are the given data in the problem, which are assumed to be known in advance:

- c_i : Cost coefficients for each decision variable, $\forall i = 1, \dots, n$. These values represent the contribution of each variable to the objective function.
- a_{ij} : Constraint coefficients, $\forall i = 1, \dots, n$ and $j = 1, \dots, m$. These define the relationships between decision variables and the constraints.
- b_j : Right-hand side values of the constraints, $\forall j = 1, \dots, m$. These represent limits or requirements for each constraint.

The general form of a linear program can be written as follows:

$$\begin{aligned}
 \max \quad & z = c_1x_1 + \cdots + c_nx_n \\
 \text{s.t.} \quad & a_{11}x_1 + \cdots + a_{1n}x_n \leq b_1 \\
 & \vdots \\
 & a_{m1}x_1 + \cdots + a_{mn}x_n \leq b_m \\
 & x_i \geq 0 \quad \forall i.
 \end{aligned} \tag{3.1}$$

Features of a Linear Program

- **Objective Function:** The linear function $z = c_1x_1 + \cdots + c_nx_n$ represents the goal to optimize. This could involve maximizing profit, minimizing cost, or achieving some other measurable outcome.
- **Constraints:** The system of inequalities (or equalities) restricts the values of the decision variables. Each constraint reflects a real-world limitation, such as resource availability or capacity limits.
- **Feasibility:** The set of all possible solutions that satisfy the constraints is called the feasible region. The optimal solution must lie within this region.
- **Linear Relationships:** Both the objective function and constraints are linear, meaning the relationships between variables are additive and proportional.
- **Sets:** For specific optimization problem, it can be useful to define sets of variables or parameters to help write down the problem in a generic formulation.

Example of a Linear Program

To illustrate, consider a specific example with three decision variables and four constraints. This includes a mix of constraint types ($\leq$, $\geq$, and $=$), showcasing the versatility of linear programming:

Decision Variables:

x_i : Continuous variables ($x_i \in \mathbb{R}$, i.e., a real number), $\forall i = 1, \dots, 3$.

Parameters (Known Input Parameters):

- c_i : Cost coefficients, $\forall i = 1, \dots, 3$.
- a_{ij} : Constraint coefficients, $\forall i = 1, \dots, 3$ and $j = 1, \dots, 4$.
- b_j : Right-hand side values for each constraint, $\forall j = 1, \dots, 4$.

The linear program can be written as:

$$\text{Minimize } z = c_1x_1 + c_2x_2 + c_3x_3 \quad (3.2)$$

$$\text{Subject to } a_{11}x_1 + a_{12}x_2 + a_{13}x_3 \geq b_1 \quad (3.3)$$

$$a_{21}x_1 + a_{22}x_2 + a_{23}x_3 \leq b_2 \quad (3.4)$$

$$a_{31}x_1 + a_{32}x_2 + a_{33}x_3 = b_3 \quad (3.5)$$

$$a_{41}x_1 + a_{42}x_2 + a_{43}x_3 \geq b_4 \quad (3.6)$$

$$x_1 \geq 0, x_2 \leq 0, x_3 \text{ unrestricted.} \quad (3.7)$$

A key feature of linear programming is the use of linear functions, defined as follows:

Definition 3.2: Linear Function

A function $z : \mathbb{R}^n \rightarrow \mathbb{R}$ is linear if there are constants $c_1, \dots, c_n \in \mathbb{R}$ such that:

$$z(x_1, \dots, x_n) = c_1x_1 + \dots + c_nx_n. \quad (3.8)$$

Linear functions are characterized by their simplicity and additivity, making them well-suited for optimization problems where proportionality is assumed. This simplicity allows linear programs to be solved efficiently, even for large-scale problems.

3.2.1. Assumptions

Inspecting Example [3.1](#) (or the more general problem in Equation [\(3.1\)](#)) we can see there are several assumptions that must be satisfied when using a linear programming model. We enumerate these below:

Proportionality Assumption A problem can be phrased as a linear program only if the contribution to the objective function *and* the left-hand-side of each constraint by each decision variable ($x_1, \dots, x_n$) is proportional to the value of the decision variable.

Additivity Assumption A problem can be phrased as a linear programming problem only if the contribution to the objective function *and* the left-hand-side of each constraint by any decision variable x_i ($i = 1, \dots, n$) is completely independent of any other decision variable x_j ($j \neq i$) and additive.

Divisibility Assumption A problem can be phrased as a linear programming problem only if the quantities represented by each decision variable are infinitely divisible (i.e., fractional answers make sense).

Certainty Assumption A problem can be phrased as a linear programming problem only if the coefficients in the objective function and constraints are known with certainty.

The first two assumptions simply assert (in English) that both the objective function and functions on the left-hand-side of the (in)equalities in the constraints are linear functions of the variables $x_1, \dots, x_n$.

The third assumption asserts that a valid optimal answer could contain fractional values for decision variables. It's important to understand how this assumption comes into play—even in the toy making example. Many quantities can be divided into non-integer values (ounces, pounds etc.) but many other quantities cannot be divided. For instance, can we really expect that it's reasonable to make $\frac{1}{2}$ a plane in the toy making example? When values must be constrained to true integer values, the linear programming problem is called an *integer programming problem*. There is a *vast* literature dealing with these problems. For many problems, particularly when the values of the decision variables may become large, a fractional optimal answer could be obtained and then rounded to the nearest integer to obtain a reasonable answer. For example, if our toy problem were re-written so that the optimal answer was to make 1045.3 planes, then we could round down to 1045.

The final assumption asserts that the coefficients (e.g., profit per plane or boat) is known with absolute certainty. In traditional linear programming, there is no lack of knowledge about the make up of the objective function, the coefficients in the left-hand-side of the constraints or the bounds on the right-hand-sides of the constraints. There is a literature on *stochastic programming* that relaxes some of these assumptions, but this too is outside the scope of the course.

Exercise 3.3

In a short sentence or two, discuss whether the problem given in Example 3.1 meets all of the assumptions of a scenario that can be modeled by a linear programming problem. Do the same for Exercise 3.1.

[Hint: Can you make $\frac{2}{3}$ of a toy? Can you run a process for $\frac{1}{3}$ of an hour?]

3.3 Examples

Outcomes

1. Identify decision variables and parameters
2. Learn how to format a linear optimization problem.
3. Identify and understand common classes of linear optimization problems.

We will begin with a few examples, and then discuss specific problem types that occur often.

Example 3.2: Production Planning

Excel PuLP Gurobipy

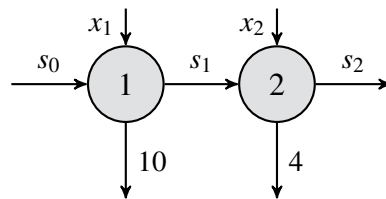
A manufacturing firm seeks to optimize its production schedule over a two-day planning horizon to meet daily demand while minimizing costs. The firm starts with an initial inventory of s_0 units. The costs associated with production and inventory are as follows:

- **Production costs:** \$10 per unit on Day 1, and \$16 per unit on Day 2.
- **Inventory cost:** \$5 per unit for carrying excess inventory from one day to the next.

Production

Inventory

Demand



The goal is to determine the number of units to produce each day and the inventory levels to minimize total costs while satisfying the demand for each day.

Solution. Sets:

- Days of production = $\{1, 2\}$.

Parameters:

- c_{p1} : Per unit production cost on Day 1 (\$10).
- c_{p2} : Per unit production cost on Day 2 (\$16).
- c_{inv} : Per unit inventory cost (\$5).
- d_1 : Demand on Day 1 (10 units).
- d_2 : Demand on Day 2 (4 units).
- s_0 : Inventory at the start of Day 1.

Decision variables:

- x_1 : Number of units produced on Day 1.
- x_2 : Number of units produced on Day 2.
- s_1 : Inventory at the end of Day 1.
- s_2 : Inventory at the end of Day 2.

Objective and Constraints:

$$\begin{array}{ll}
 \min & z = 10x_1 + 16x_2 + 5s_1 + 5s_2 & \text{(Total cost)} \\
 \text{s.t.} & s_0 + x_1 = 10 + s_1 & \text{(Day 1 balance)} \\
 & s_1 + x_2 = 4 + s_2 & \text{(Day 2 balance)} \\
 & x_1, x_2, s_0, s_1, s_2 \geq 0 & \text{(Non-negativity constraints).}
 \end{array}$$

**Definition 3.4: Optimization model components**

When defining an optimization model, it is essential to identify and clearly articulate the following five components:

- **Sets:** What lists of data do you need to write down? These could include indices, categories, or groups relevant to the problem.
- **Parameters (Data):** What is the actual data for the problem? These are the fixed numerical values such as costs, capacities, or resource availabilities.
- **Decision Variables:** What decisions need to be made to provide a solution? These variables represent the quantities that can be controlled or adjusted, such as production levels or resource allocations.
- **Objective Function:** The expression that we want to maximize or minimize. This is the goal of the optimization, such as minimizing cost or maximizing profit.
- **Constraints:** Certain rules that those decision variables should satisfy. These include limitations or requirements, such as resource capacities or demand fulfillment.

Each of these components plays a crucial role in formulating a well-structured optimization model that accurately represents the problem and provides actionable solutions. The role of Sets will become more important as we get into more complicated optimization models.

Example 3.3: Production with welding robot

[Excel](#) [PuLP](#) [Gurobipy](#)

You have 21 units of transparent aluminum alloy (TAA), LazWeld1, a joining robot leased for 23 hours, and CrumCut1, a cutting robot leased for 17 hours of aluminum cutting. You also have production code for a bookcase, desk, and cabinet, along with commitments to buy any of these you can produce for \$18, \$16, and \$10 apiece, respectively. A bookcase requires 2 units of TAA, 3 hours of joining, and 1 hour of cutting, a desk requires 2 units of TAA, 2 hours of joining, and 2 hour of cutting, and a cabinet requires 1 unit of TAA, 2 hours of joining, and 1 hour of cutting.

Formulate an LP to maximize your revenue given your current resources.

Solution.

Sets:

- The types of objects = { bookcase, desk, cabinet }.

Parameters:

- Purchase cost of each object
- Units of TAA needed for each object
- Hours of joining needed for each object
- Hours of cutting needed for each object
- Hours of TAA, Joining, and Cutting available on robots

Decision variables:

x_i : number of units of product i to produce,
for all i =bookcase, desk, cabinet.

Objective and Constraints:

$$\begin{aligned}
 \max \quad & z = 18x_1 + 16x_2 + 10x_3 && \text{(profit)} \\
 \text{s.t.} \quad & 2x_1 + 2x_2 + 1x_3 \leq 21 && \text{(TAA)} \\
 & 3x_1 + 2x_2 + 2x_3 \leq 23 && \text{(LazWeld1)} \\
 & 1x_1 + 2x_2 + 1x_3 \leq 17 && \text{(CrumCut1)} \\
 & x_1, x_2, x_3 \geq 0.
 \end{aligned}$$



Example 3.4: The Diet Problem

Excel | PuLP | Gurobipy

In the future (as envisioned in a bad 70's science fiction film) all food is in tablet form, and there are four types, green, blue, yellow, and red. A balanced, futuristic diet requires, at least 20 units of Iron, 25 units of Vitamin B, 30 units of Vitamin C, and 15 units of Vitamin D. Formulate an LP that ensures a balanced diet at the minimum possible cost.

Tablet	Iron	B	C	D	Cost (\$)
Chem 1	6	6	7	4	1.25
Chem 2	4	5	4	9	1.05
Chem 3	5	2	5	6	0.85
Chem 4	3	6	3	2	0.65

Solution. Sets:

- Set of tablets {1,2,3,4}

Parameters:

- Iron in each tablet
- Vitamin B in each tablet
- Vitamin C in each tablet
- Vitamin D in each tablet
- Cost of each tablet

Decision variables:

x_i : number of tablet of type i to include in the diet, $\forall i \in \{1, 2, 3, 4\}$.

Objective and Constraints:

$$\begin{array}{ll}
 \text{Min} & z = 1.25x_1 + 1.05x_2 + 0.85x_3 + 0.65x_4 \\
 \text{s.t.} & 6x_1 + 4x_2 + 5x_3 + 3x_4 \geq 20 \\
 & 6x_1 + 5x_2 + 2x_3 + 6x_4 \geq 25 \\
 & 7x_1 + 4x_2 + 5x_3 + 3x_4 \geq 30 \\
 & 4x_1 + 9x_2 + 6x_3 + 2x_4 \geq 15 \\
 & x_1, x_2, x_3, x_4 \geq 0.
 \end{array}$$

**Example 3.5: The Next Diet Problem**
[Excel](#) [PuLP](#) [Gurobipy](#)

Progress is important, and our last problem had too many tablets, so we are going to produce a single, purple, 10 gram tablet for our futuristic diet requires, which are at least 20 units of Iron, 25 units of Vitamin B, 30 units of Vitamin C, and 15 units of Vitamin D, and 2000 calories. The tablet is made from blending 4 nutritious chemicals; the following table shows the units of our nutrients per, and cost of, grams of each chemical.

Formulate an LP that ensures a balanced diet at the minimum possible cost.

Tablet	Iron	B	C	D	Calories	Cost (\$)
Chem 1	6	6	7	4	1000	1.25
Chem 2	4	5	4	9	250	1.05
Chem 3	5	2	5	6	850	0.85
Chem 4	3	6	3	2	750	0.65

Solution.Sets:

- Set of chemicals $\{1, 2, 3, 4\}$

Parameters:

- Iron in each chemical
- Vitamin B in each chemical
- Vitamin C in each chemical
- Vitamin D in each chemical
- Cost of each chemical

Decision variables:

x_i : grams of chemical i to include in the purple tablet, $\forall i = 1, 2, 3, 4$.

Objective and Constraints:

$$\begin{aligned}
 \min \quad & z = 1.25x_1 + 1.05x_2 + 0.85x_3 + 0.65x_4 \\
 \text{s.t.} \quad & 6x_1 + 4x_2 + 5x_3 + 3x_4 \geq 20 \\
 & 6x_1 + 5x_2 + 2x_3 + 6x_4 \geq 25 \\
 & 7x_1 + 4x_2 + 5x_3 + 3x_4 \geq 30 \\
 & 4x_1 + 9x_2 + 6x_3 + 2x_4 \geq 15 \\
 & 1000x_1 + 250x_2 + 850x_3 + 750x_4 \geq 2000 \\
 & x_1 + x_2 + x_3 + x_4 = 10 \\
 & x_1, x_2, x_3, x_4 \geq 0.
 \end{aligned}$$



Example 3.6: Work Scheduling Problem

[Excel](#) [PuLP](#) [Gurobipy](#)

You are the manager of LP Burger. The following table shows the minimum number of employees required to staff the restaurant on each day of the week. Each employees must work for five consecutive days. Formulate an LP to find the minimum number of employees required to staff the restaurant.

Day of Week	Workers Required
1 = Monday	6
2 = Tuesday	4
3 = Wednesday	5
4 = Thursday	4
5 = Friday	3
6 = Saturday	7
7 = Sunday	7

Solution. This problem has multiple optimal solutions for which days workers begin working on, all of which result in 8 total workers hired.

Decision variables:

x_i : the number of workers that start 5 consecutive days of work on day i , $i = 1, \dots, 7$

Objective and Constraints:

$$\begin{array}{ll}
 \text{Min} & z = x_1 + x_2 + x_3 + x_4 + x_5 + x_6 + x_7 \\
 \text{s.t.} & x_1 + x_4 + x_5 + x_6 + x_7 \geq 6 \\
 & x_2 + x_5 + x_6 + x_7 + x_1 \geq 4 \\
 & x_3 + x_6 + x_7 + x_1 + x_2 \geq 5 \\
 & x_4 + x_7 + x_1 + x_2 + x_3 \geq 4 \\
 & x_5 + x_1 + x_2 + x_3 + x_4 \geq 3 \\
 & x_6 + x_2 + x_3 + x_4 + x_5 \geq 7 \\
 & x_7 + x_3 + x_4 + x_5 + x_6 \geq 7 \\
 & x_1, x_2, x_3, x_4, x_5, x_6, x_7 \geq 0.
 \end{array}$$

One solution is as follows:

LP Solution	IP Solution
$z_{LP} = 7.333$	$z_I = 8.0$
$x_1 = 0$	$x_1 = 0$
$x_2 = 0.333$	$x_2 = 0$
$x_3 = 1$	$x_3 = 0$
$x_4 = 2.333$	$x_4 = 3$
$x_5 = 0$	$x_5 = 0$
$x_6 = 3.333$	$x_6 = 4$
$x_7 = 0.333$	$x_7 = 1$



Example 3.7: LP Burger - extended

Excel PuLP Gurobipy

LP Burger has changed its policy, and allows, at most, two part time workers, who work for two consecutive days in a week. Formulate this problem.

Solution. Decision variables:

x_i : the number of workers that start 5 consecutive days of work on day i , $i = 1, \dots, 7$

y_i : the number of workers that start 2 consecutive days of work on day i , $i = 1, \dots, 7$.

Objective and Constraints:

$$\begin{aligned}
 &\text{Min} && z = 5(x_1 + x_2 + x_3 + x_4 + x_5 + x_6 + x_7) \\
 &&& + 2(y_1 + y_2 + y_3 + y_4 + y_5 + y_6 + y_7) \\
 &\text{s.t.} && x_1 + x_4 + x_5 + x_6 + x_7 + y_1 + y_7 \geq 6 \\
 &&& x_2 + x_5 + x_6 + x_7 + x_1 + y_2 + y_1 \geq 4 \\
 &&& x_3 + x_6 + x_7 + x_1 + x_2 + y_3 + y_2 \geq 5 \\
 &&& x_4 + x_7 + x_1 + x_2 + x_3 + y_4 + y_3 \geq 4 \\
 &&& x_5 + x_1 + x_2 + x_3 + x_4 + y_5 + y_4 \geq 3 \\
 &&& x_6 + x_2 + x_3 + x_4 + x_5 + y_6 + y_5 \geq 7 \\
 &&& x_7 + x_3 + x_4 + x_5 + x_6 + y_7 + y_6 \geq 7 \\
 &&& y_1 + y_2 + y_3 + y_4 + y_5 + y_6 + y_7 \leq 2 \\
 &&& x_i \geq 0, y_i \geq 0, \forall i = 1, \dots, 7.
 \end{aligned}$$



3.3.1. Knapsack Problem

We consider now a simple example where the variables can only take the values of 0 or 1. We call these *binary variables*.

Example 3.8: Camping Trip

[Excel](#) [PuLP](#) [Gurobipy](#)

Imagine you are preparing for a week-long camping trip to the mountains. You have a backpack with a weight capacity of 20 kilograms. Your goal is to pack the most valuable items to ensure a comfortable and safe trip without exceeding the backpack's weight limit. Each item has a weight and a value associated with it, representing its importance and utility for the trip. The table below lists the potential items you can take, their weights in kilograms, and their values on a scale from 1 to 10 (with 10 being the most valuable).

Formulate an Integer Program to find the optimal items you should bring on your trip with you.

Item	Index	Weight (kg)	Value
Tent	A	4	10
Stove	B	2	9
Sleeping Bag	C	3	8
First Aid Kit	D	1	7
Food Supplies	E	5	9
Water Filter	F	1	6
Clothes	G	4	5
Map and Compass	H	0.5	7

Solution. Decision variables:

x_i : whether to include item i in the backpack, where $i \in \{A, B, C, D, E, F, G, H\}$, $x_i = 1$ if item i is included, and $x_i = 0$ otherwise.

Objective and Constraints:

$$\begin{aligned}
 \text{Max } z &= 10x_A + 9x_B + 8x_C + 7x_D + 9x_E + 6x_F + 5x_G + 7x_H \\
 \text{s.t. } &4x_A + 2x_B + 3x_C + 1x_D + 5x_E + 1x_F + 4x_G + 0.5x_H \leq 20 \\
 &x_A, x_B, x_C, x_D, x_E, x_F, x_G, x_H \in \{0, 1\}.
 \end{aligned}$$



3.3.2. Capital Investment

We next see a similar application of binary variables.

Example 3.9: Capital Allocation Problem

Excel PuLP Gurobipy

You are a financial planner for an investment firm. The firm has \$100,000 to invest in a portfolio of projects. Each project has a projected return and requires an initial investment. The goal is to maximize the total return of the portfolio while not exceeding the available capital. The following table lists potential projects, their required investments, and their projected returns.

Formulate an LP to maximize the total return of the portfolio while not exceeding the available investment capital.

Project	Investment Required (\$)	Projected Return (\$)
A	20,000	30,000
B	30,000	40,000
C	50,000	75,000
D	10,000	15,000
E	40,000	60,000

Solution.

Decision variables:

x_i : whether to include project i in the portfolio, where $i \in \{A, B, C, D, E\}$, $x_i = 1$ if project i is included, and $x_i = 0$ otherwise.

Objective and Constraints:

$$\begin{aligned}
 \text{Max } z &= 30,000x_A + 40,000x_B + 75,000x_C + 15,000x_D + 60,000x_E \\
 \text{s.t. } &20,000x_A + 30,000x_B + 50,000x_C + 10,000x_D + 40,000x_E \leq 100,000 \\
 &x_A, x_B, x_C, x_D, x_E \in \{0, 1\}.
 \end{aligned}$$



3.3.3. Assignment Problem

Consider the assignment of 3 employees to 3 tasks, where each employee has a cost to do each task. How to figure out the best assignment to the tasks?

Example 3.10: Hiring for tasks
[Excel](#) [PuLP](#) [Gurobipy](#)

In this assignment problem, we need to hire three people (Person 1, Person 2, Person 3) to three tasks (Task 1, Task 2, Task 3). In the table below, we list the cost of hiring each person for each task, in dollars. Since each person has a different cost for each task, we must make an assignment to minimize

	Cost	Task 1	Task 2	Task 3
our total cost.	Person 1	40	47	80
	Person 2	72	36	58
	Person 3	24	61	71

Given the specific costs of assigning three people to three tasks, we can write out the mathematical model explicitly using the given numbers.

Solution.

Sets: Set of tasks and set of persons

Parameters: Cost c_{ij} for person i to complete task j .

Variables: Let $x_{ij} = 1$ if person i is assigned to task j and 0 otherwise.

Objective Function:

Minimize the total cost of assignments:

$$z = 40x_{11} + 47x_{12} + 80x_{13} + 72x_{21} + 36x_{22} + 58x_{23} + 24x_{31} + 61x_{32} + 71x_{33} \quad (3.1)$$

Subject to Constraints:

Each person is assigned to exactly one task:

$$x_{11} + x_{12} + x_{13} = 1 \quad (3.2)$$

$$x_{21} + x_{22} + x_{23} = 1 \quad (3.3)$$

$$x_{31} + x_{32} + x_{33} = 1 \quad (3.4)$$

Each task is assigned to exactly one person:

$$x_{11} + x_{21} + x_{31} = 1 \quad (3.5)$$

$$x_{12} + x_{22} + x_{32} = 1 \quad (3.6)$$

$$x_{13} + x_{23} + x_{33} = 1 \quad (3.7)$$

Binary constraints on the variables:

$$x_{ij} \in \{0, 1\} \quad \forall i \in \{1, 2, 3\}, \forall j \in \{1, 2, 3\} \quad (3.8)$$



3.4 Exercises

Exercise 3.5

Which ones are linear functions?

1. $x^2 + 3y + 2z$
2. $\sqrt{x_1 x_2 x_3}$
3. $\sum_{i=1}^n a_i 2^{x_i}$, where $a_1, a_2, \dots, a_n$ are constant
4. $\sum_{i=1}^n a_i x_i$, where $a_1, a_2, \dots, a_n$ are constant

Exercise 3.6

Which one is a linear function?

1. $2x_1 + x_2 x_3 - 7x_3$
2. $\sqrt{2}x_1 + x_2 + 3^{1.6}x_3$

Exercise 3.7

Which ones are linear inequalities? Can any be made into linear inequalities?

1. $\frac{x_1}{x_1 + x_2 + x_3} \geq 0.5$
2. $\sin x_1 + x_2 + x_3 = 0.5$
3. $\sum_{i=1}^n a_i 2^{x_i} \geq b$, where $a_1, a_2, \dots, a_n, b$ are constant
4. $\sum_{i=1}^n a_i x_i = b$, where $a_1, a_2, \dots, a_n, b$ are constant
5. $\sum_{i=1}^n a_i x_i \geq b$, where $a_1, a_2, \dots, a_n, b$ are constant

4. Software - Excel

Outcomes

- *Learn relevant excel functions*
- *Setup linear programs in excel*
- *Solve linear programs in excel*

4.1 Using Excel Solver

Excel Solver is a powerful tool for solving optimization problems with linear and nonlinear constraints. Before diving into using Solver, it is essential to understand some basic Excel functions and techniques that are commonly used when setting up optimization problems. These functions help organize data, compute intermediate values, and define objective functions and constraints.

4.1.1. Useful Excel Functions and Techniques

USING FORMULAS IN CELLS In Excel, formulas begin with an equals sign (=). For example, typing =A1+B1 in a cell will calculate the sum of the values in cells A1 and B1.

SUM() The SUM() function adds a range of numbers. For example, =SUM(A1:A10) calculates the sum of all values in the range A1 to A10.

SUMPRODUCT() The SUMPRODUCT() function multiplies corresponding elements in two or more arrays and returns the sum of those products. This is particularly useful for computing dot products or weighted sums. For example, =SUMPRODUCT(A1:A10, B1:B10) computes $\sum_{i=1}^{10} A_i B_i$.

COUNT() The COUNT() function counts the number of numeric entries in a range. For example, =COUNT(A1:A10) returns the number of numeric cells in the range A1 to A10.

COUNTIF() The COUNTIF() function counts the number of cells in a range that meet a specific condition. For example, =COUNTIF(A1:A10, ">5") counts the number of cells in A1:A10 with values greater than 5.

DUPLICATING FORMULAS ACROSS ROWS OR COLUMNS To apply a formula to multiple cells, you can drag the fill handle (a small square at the bottom-right corner of the selected cell) across the desired range. Excel automatically adjusts cell references based on the relative position (relative referencing). To keep a reference constant, use the dollar sign (\$) in the cell reference (absolute referencing). For example, \$A\$1 always refers to cell A1.

CONDITIONAL FORMATTING Conditional formatting allows you to highlight cells that meet specific criteria. For example, you can format cells with values above a threshold in bold or a particular color to identify important data visually.

DATA VALIDATION Data validation helps restrict the type of data entered in a cell. For example, you can set a rule that only allows numbers within a specific range or text from a predefined list.

NAMING RANGES Naming ranges makes formulas easier to read and maintain. Instead of =SUM(A1:A10), you can name the range "Sales" and write =SUM(Sales).

USING LOGICAL FUNCTIONS Logical functions like IF(), AND(), and OR() are helpful for decision-making within formulas. For example, =IF(A1>10, High, Low) returns High if A1 is greater than 10, and Low otherwise.

With these foundational tools, setting up optimization problems becomes much more manageable. In the next section, we will explore how to use these functions in combination with Excel Solver to define and solve optimization models.

Logical functions like IF(), AND(), and OR() are helpful for decision-making within formulas. For example, =IF(A1>10, High, Low) returns High if A1 is greater than 10, and Low otherwise.

With these foundational tools, setting up optimization problems becomes much more manageable. In the next section, we will explore how to use these functions in combination with Excel Solver to define and solve optimization models.

4.1.2. How to Use Excel Solver

Excel Solver is a powerful add-in that allows users to define and solve optimization problems. This section provides a step-by-step guide on how to effectively layout data and use Solver for optimization.

STEP 1: LAYOUT THE DATA Organizing your data clearly is crucial for using Solver effectively. Use color coding to distinguish between different types of cells:

- **Data:** Cells containing fixed input values, such as coefficients or constants, should be highlighted in light blue.
- **Variables:** Cells representing decision variables should be highlighted in yellow.
- **Formulas:** Cells with formulas, such as the objective function or summarizing constraints values should be highlighted in light green.

Other color combinations are also acceptable, as long as they consistently distinguish between these components.

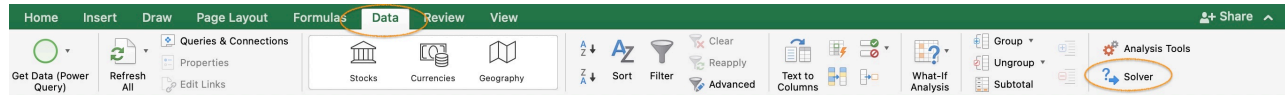
Objective	=SUMPRODUCT(B6:C6,B8:C8)					
Variables	x1	x2				
obj. coeff	7	6				
			Resources Used			
constraint 1	3	1	=SUMPRODUCT(\$B\$6:\$C\$6,B11:C11)	<=	120	
constraint 2	1	2	=SUMPRODUCT(\$B\$6:\$C\$6,B12:C12)	<=	160	
constraint 3	1	0	=SUMPRODUCT(\$B\$6:\$C\$6,B13:C13)	<=	35	

STEP 2: DEFINE THE OBJECTIVE FUNCTION Identify the cell that represents the objective function. This cell should contain a formula that calculates the value to be maximized or minimized, such as total profit or cost.

STEP 3: SET UP CONSTRAINTS Clearly specify the constraints of the problem. Constraints can be represented using formulas that involve the variables, coefficients, and right-hand side values. For example, a constraint ensuring that total production does not exceed capacity can be written as `=SUMPRODUCT(Production, Coefficients) <= Capacity`.

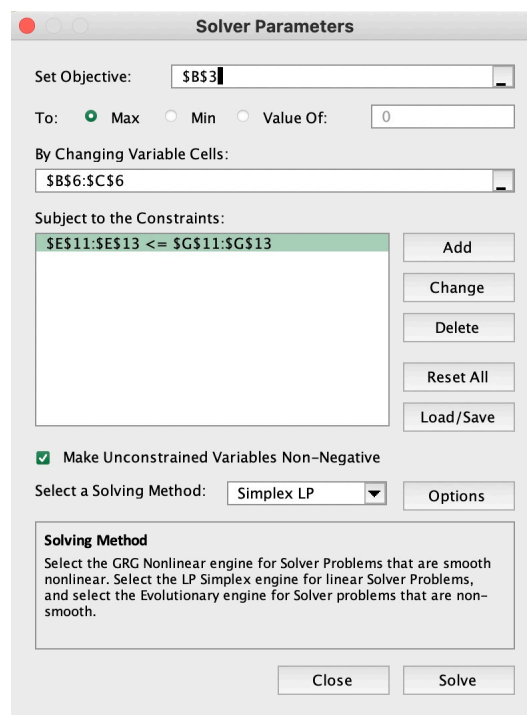
STEP 4: OPEN SOLVER Ensure Solver is enabled in Excel. If not, you can activate it by navigating to **File > Options > Add-Ins > Manage Excel Add-ins** and checking the box for Solver.

Once Solver is enabled, go to **Data > Solver** to open the Solver Parameters dialog box.



STEP 5: DEFINE SOLVER PARAMETERS In the Solver Parameters dialog box:

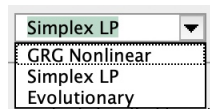
- Set the **Objective** to the cell containing the objective function.
- Choose **Max** or **Min** to indicate whether the objective is to be maximized or minimized.
- Specify the **Variable Cells** by selecting the range of cells representing decision variables.
- Add **Constraints** by specifying the relationship (e.g., `<=`, `>=`, `=`) and the cells involved.
- When the variables are non-negative, you can select **Make Unconstrained Variables Non-Negative**.



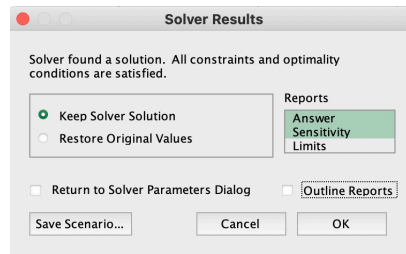
STEP 6: CHOOSE A SOLVING METHOD Solver provides three solving methods:

- **GRG Nonlinear:** For smooth nonlinear problems.
- **Simplex LP:** For linear programming problems.
- **Evolutionary:** For non-smooth problems or those with integer constraints.

Select the method that matches the problem type.



STEP 7: SOLVE THE PROBLEM Click **Solve** to run Solver. If Solver finds a solution, it will display the results and allow you to keep or discard them. Analyze the solution to ensure it meets the problem's requirements.



STEP 8: INTERPRET THE RESULTS Review the values of the decision variables and the objective function. Check that all constraints are satisfied and assess the solution's feasibility and quality.

- Answer report

	A	B	C	D	E	F	G															
1	Microsoft Excel 16.77 Answer Report																					
2	Worksheet: [toymaker (1).xlsx]Sheet1																					
3	Report Created: 1/17/25 4:27:45 AM																					
4	Result: Solver found a solution. All constraints and optimality conditions are satisfied.																					
5	Solver Engine																					
9	Solver Options																					
12																						
13																						
14	Objective Cell (Max)																					
15	<table><tr><th>Cell</th><th>Name</th><th>Original Value</th><th>Final Value</th></tr><tr><td>\$B\$3</td><td>Objective</td><td>0</td><td>544</td></tr></table>							Cell	Name	Original Value	Final Value	\$B\$3	Objective	0	544							
Cell	Name	Original Value	Final Value																			
\$B\$3	Objective	0	544																			
17																						
18																						
19	Variable Cells																					
20	<table><tr><th>Cell</th><th>Name</th><th>Original Value</th><th>Final Value</th><th>Integer</th></tr><tr><td>\$B\$6</td><td>x1</td><td>0</td><td>16</td><td>Contin</td></tr><tr><td>\$C\$6</td><td>x2</td><td>0</td><td>72</td><td>Contin</td></tr></table>							Cell	Name	Original Value	Final Value	Integer	\$B\$6	x1	0	16	Contin	\$C\$6	x2	0	72	Contin
Cell	Name	Original Value	Final Value	Integer																		
\$B\$6	x1	0	16	Contin																		
\$C\$6	x2	0	72	Contin																		
23																						
24																						
25	Constraints																					
26	<table><tr><th>Cell</th><th>Name</th><th>Cell Value</th><th>Formula</th><th>Status</th><th>Slack</th></tr><tr><td colspan="6">\$E\$11:\$E\$13 <= \$G\$11:\$G\$13</td></tr></table>							Cell	Name	Cell Value	Formula	Status	Slack	\$E\$11:\$E\$13 <= \$G\$11:\$G\$13								
Cell	Name	Cell Value	Formula	Status	Slack																	
\$E\$11:\$E\$13 <= \$G\$11:\$G\$13																						
27																						

- Sensitivity report

	A	B	C	D	E	F	G	H
1	Microsoft Excel 16.77 Sensitivity Report							
2	Worksheet: [toymaker (1).xlsx]Sheet1							
3	Report Created: 1/17/25 4:27:46 AM							
4								
5								
6	Variable Cells							
7				Final	Reduced	Objective	Allowable	Allowable
8	Cell	Name		Value	Cost	Coefficient	Increase	Decrease
9	\$B\$6	x1		16	0	7	11	4
10	\$C\$6	x2		72	0	6	8	3.666666667
11								
12	Constraints							
13				Final	Shadow	Constraint	Allowable	Allowable
14	Cell	Name		Value	Price	R.H. Side	Increase	Decrease
15	\$E\$11:\$E\$13 <= \$G\$11:\$G\$13							
16								
17								
18								
19								

- Solution on spreadsheet

	A	B	C	D	E	F	G
1							
2							
3	Objective	544					
4							
5	Variables	x1	x2				
6		16	72				
7							
8	obj. coeff	7	6				
9							
10					Resources Used		
11	constraint 1	3	1		120	<=	120
12	constraint 2	1	2		160	<=	160
13	constraint 3	1	0		16	<=	35
14							

Resources

Examples and guides

- [Installing Excel Solver on Windows](#)
- <https://www.excel-easy.com/data-analysis/solver.html>
- [Excel Solver - Introduction on Youtube](#)
- [Some notes from MIT](#)

Some videos

- [Solving a linear program](#)
- [Optimal product mix](#)
- [Set Cover](#)
- [Introduction to Designing Optimization Models Using Excel Solver](#)
- [Traveling Salesman Problem](#)
- [Also Travelin Salesman Problem](#)
- [Multiple Traveling Salesman Problem](#)
- [Shortest Path](#)

Some other links

- [Loan Example](#)
- [Several Examples including TSP](#)

5. Graphically Solving Linear Programs

Outcomes

- A. Learn how to plot the feasible region and the objective function.
- B. Identify and compute extreme points of the feasible region.
- C. Find the optimal solution(s) to a linear program graphically.
- D. Classify the type of result of the problem as infeasible, unbounded, unique optimal solution, or infinitely many optimal solutions.

Linear Programs (LP's) with two variables can be solved graphically by plotting the feasible region along with the level curves of the objective function.¹ We will show that we can find a point in the feasible region that maximizes the objective function using the level curves of the objective function.

We will begin with an easy example that is bounded and investigate the structure of the feasible region. We will then explore other examples.

5.1 Nonempty and Bounded Problem

Consider the problem

$$\begin{array}{ll}\max & 2x_1 + 5x_2 \\ \text{s.t.} & x_1 + 2x_2 \leq 16 \\ & 5x_1 + 3x_2 \leq 45 \\ & x_1, x_2 \geq 0\end{array}$$

We want to start by plotting the *feasible region*, that is, the set points (x_1, x_2) that satisfy all the constraints.

We can plot this by first plotting the four lines

- $x_1 + 2x_2 = 16$
- $5x_1 + 3x_2 = 45$
- $x_1 = 0$
- $x_2 = 0$

¹Special thanks to Joshua Emmanuel and Christopher Griffin for sharing their content to help put this section together. Proper citations and referenes are forthcoming.

and then shading in the side of the space cut out by the corresponding inequality.

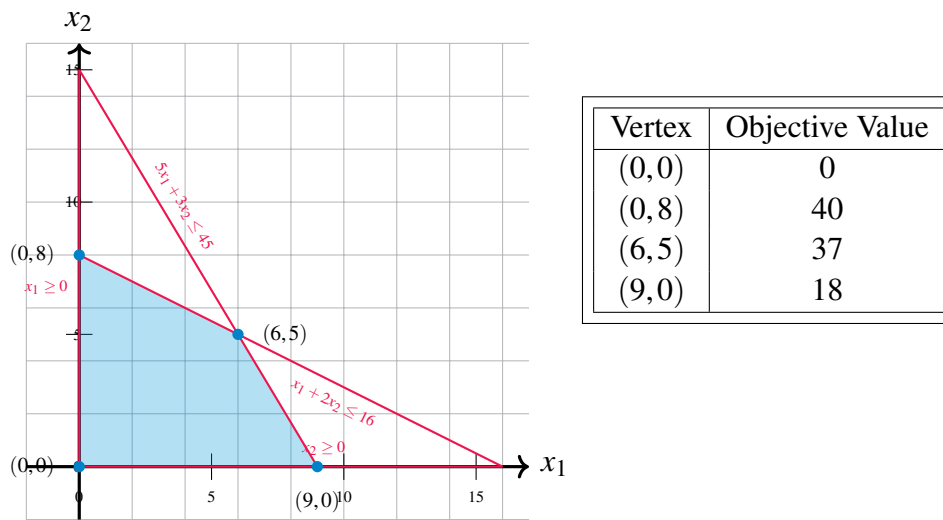


Figure 5.1: Desmos: Interactive plot!

The resulting feasible region can then be shaded in as the region that satisfies all the inequalities.

Notice that the feasible region is nonempty (it has points that satisfy all the inequalities) and also that it is bounded (the feasible points don't continue infinitely in any direction).

We want to identify the *extreme points* (i.e., the corners) of the feasible region. Understanding these points will be critical to understanding the optimal solutions of the model. Notice that all extreme points can be computed by finding the intersection of 2 of the lines. But! Not all intersections of any two lines are feasible.

We will later use the terminology *basic feasible solution* for an extreme point of the feasible region, and *basic solution* as a point that is the intersection of 2 lines, but is actually infeasible (does not satisfy all the constraints).

Theorem 5.1: Optimal Solution at a Vertex

If the feasible region of the linear program is nonempty and bounded, then there exists an optimal solution at vertex of the feasible region.

This theorem implies the following algorithm.

Algorithm 1 Enumerate Vertices Algorithm for Solving a Linear Programming Problem

1. **Identify the Feasible Region:** Determine the set of constraints defining the feasible region in the problem.
2. **Enumerate Vertices:** Find all the vertices (corner points) of the feasible region. This can be done by solving pairs of constraint equations to find their intersections, keeping only those that satisfy all constraints.
3. **Compute Objective Values:** Evaluate the objective function at each vertex of the feasible region.
4. **Compare Objective Values:** For a maximization problem, identify the vertex with the highest objective value. For a minimization problem, identify the vertex with the lowest objective value.
5. **Determine the Optimal Solution:** The vertex with the optimal objective value is the solution to the linear programming problem.

We will explore why Theorem 5.1 is true, and also what happens when the feasible region does not satisfy the assumptions of either nonempty or bounded.

5.2 Graphical Approach

We illustrate the idea first using the problem from Example 3.1. In a later section, we will formalize the mathematics to explain this result.

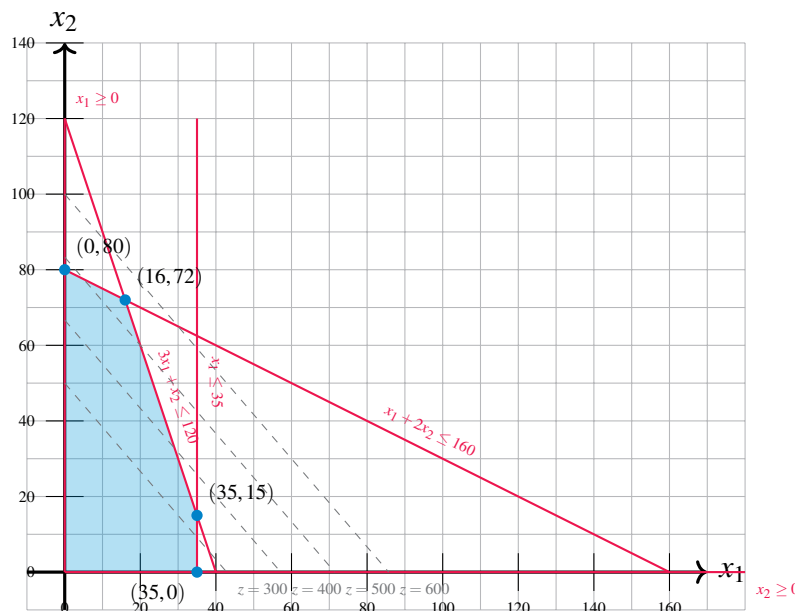


Figure 5.2: Feasible Region and Level Curves of the Objective Function: The shaded region in the plot is the feasible region and represents the intersection of the five inequalities constraining the values of x_1 and x_2 . On the right, we see the optimal solution is the “last” point in the feasible region that intersects a level set as we move in the direction of increasing profit.

Example 5.2: Continuation of ToyMaker Example

Let's continue the example of the Toy Maker begin in Example 3.1. Solve this problem graphically.

Solution. To solve the linear programming problem graphically, begin by drawing the feasible region. This is shown in the blue shaded region of Figure 5.2.

After plotting the feasible region, the next step is to plot the level curves of the objective function. In our problem, the level sets will have the form:

$$7x_1 + 6x_2 = z \implies x_2 = \frac{-7}{6}x_1 + \frac{z}{6}$$

This is a set of parallel lines with slope $-7/6$ and intercept $z/6$ where z can be varied as needed. The level curves for various values of z are parallel lines. In Figure 5.2 they are shown in colors ranging from red to yellow depending upon the value of z . Larger values of z are more yellow.

To solve the linear programming problem, follow the level sets along the gradient (shown as the black arrow) until the last level set (line) intersects the feasible region. If you are doing this by hand, you can draw a single line of the form $7x_1 + 6x_2 = z$ and then simply draw parallel lines in the direction of the gradient $(7, 6)$. At some point, these lines will fail to intersect the feasible region. The last line to intersect the feasible region will do so at a point that maximizes the profit. In this case, the point that maximizes $z(x_1, x_2) = 7x_1 + 6x_2$, subject to the constraints given, is $(x_1^*, x_2^*) = (16, 72)$.

Note the point of optimality $(x_1^*, x_2^*) = (16, 72)$ is at a corner of the feasible region. This corner is formed by the intersection of the two lines: $3x_1 + x_2 = 120$ and $x_1 + 2x_2 = 160$. In this case, the constraints

$$3x_1 + x_2 \leq 120$$

$$x_1 + 2x_2 \leq 160$$

are both *binding*, while the other constraints are non-binding. In general, we will see that when an optimal solution to a linear programming problem exists, it will always be at the intersection of several binding constraints; that is, it will occur at a corner of a higher-dimensional polyhedron. ♠

We can now define an algorithm for identifying the solution to a linear programming problem in two variables with a *bounded* feasible region.

(First Try) Algorithm for Solving a Linear Programming Problem Graphically

Bounded Feasible Region, Unique Solution

1. Plot the feasible region defined by the constraints.
 2. Plot the level sets of the objective function.
 3. For a maximization problem, identify the level set corresponding the greatest (least, for minimization) objective function value that intersects the feasible region. This point will be at a corner.
 4. The point on the corner intersecting the greatest (least) level set is a solution to the linear programming problem.
-

Example 5.3: Lemonade Vendor

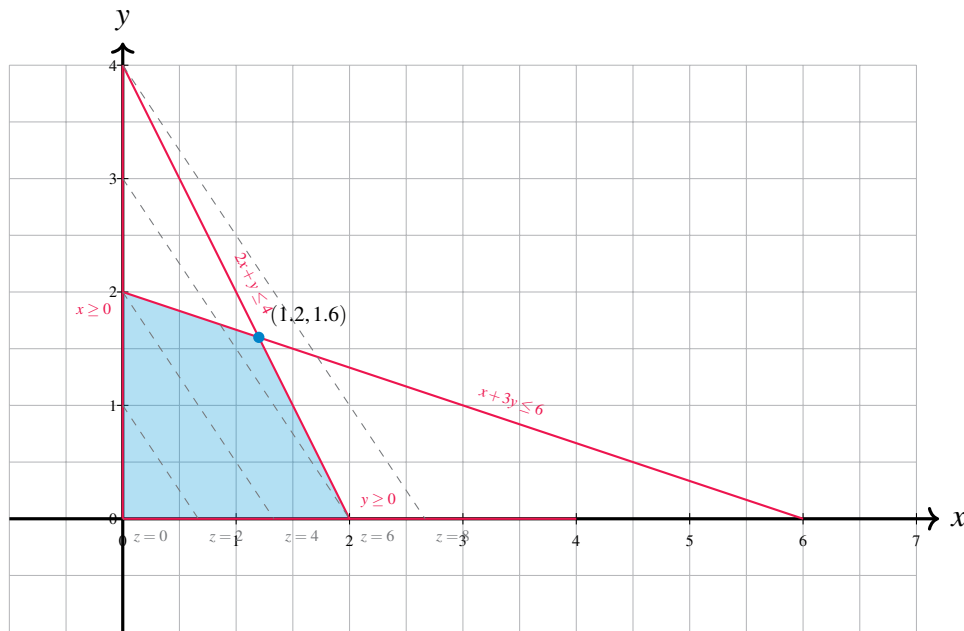
Say you are a vendor of lemonade and lemon juice. Each unit of lemonade requires 1 lemon and 2 litres of water. Each unit of lemon juice requires 3 lemons and 1 litre of water. Each unit of lemonade gives a profit of three dollars. Each unit of lemon juice gives a profit of two dollars. You have 6 lemons and 4 litres of water available. How many units of lemonade and lemon juice should you make to maximize profit?

If we let x denote the number of units of lemonade to be made and let y denote the number of units of lemon juice to be made, then the profit is given by $3x + 2y$ dollars. We call $3x + 2y$ the objective function. Note that there are a number of constraints that x and y must satisfy. First of all, x and y should be nonnegative. The number of lemons needed to make x units of lemonade and y units of lemon juice is $x + 3y$ and cannot exceed 6. The number of litres of water needed to make x units of lemonade and y units of lemon juice is $2x + y$ and cannot exceed 4. Hence, to determine the maximum profit, we need to maximize $3x + 2y$ subject to x and y satisfying the constraints $x + 3y \leq 6$, $2x + y \leq 4$, $x \geq 0$, and $y \geq 0$.

A more compact way to write the problem is as follows:

$$\begin{array}{llll} \text{maximize} & 3x & + & 2y \\ \text{subject to} & x & + & 3y \leq 6 \\ & 2x & + & y \leq 4 \\ & x & & \geq 0 \\ & & & y \geq 0. \end{array}$$

Solution. We can solve this maximization problem graphically as follows. We first sketch the set of (x, y) satisfying the constraints, called the feasible region, on the (x, y) -plane. We then take the objective function $3x + 2y$ and turn it into an equation of a line $3x + 2y = z$ where z is a parameter. Note that as the value of z increases, the line defined by the equation $3x + 2y = z$ moves in the direction of the normal vector $(3, 2)$. We call this direction the direction of improvement. Determining the maximum value of the objective function, called the optimal value, subject to the constraints amounts to finding the maximum value of z so that the line defined by the equation $3x + 2y = z$ still intersects the feasible region.



In the figure above, the lines with z at 0, 4 and 6.8 have been drawn. From the picture, we can see that if z is greater than 6.8, the line defined by $3x + 2y = z$ will not intersect the feasible region. Hence, the profit cannot exceed 6.8 dollars.

As the line $3x + 2y = 6.8$ does intersect the feasible region, 6.8 is the maximum value for the objective function. Note that there is only one point in the feasible region that intersects the line $3x + 2y = 6.8$, namely $(x, y) = (1.2, 1.6)$. In other words, to maximize profit, we want to make 1.2 units of lemonade and 1.6 units of lemon juice. ♠

The example linear programming problem presented in this has a single optimal solution. In general, the following outcomes can occur in solving a linear programming problem:

1. The linear programming problem has a unique solution. (We've already seen this.)
2. There are infinitely many alternative optimal solutions.
3. There is no solution and the problem's objective function can grow to positive infinity for maximization problems (or negative infinity for minimization problems).
4. There is no solution to the problem at all.

Case 3 above can only occur when the feasible region is unbounded; that is, it cannot be surrounded by a ball with finite radius. We will illustrate each of these possible outcomes in the next four sections. We will prove that this is true in a later chapter.